Paper Mock 1: answers and marking

Every answer below was checked with GHC 9 (types with `:t`, values by running them). Other correct answers get full marks too.

Question 1: Types

	Expression	Type
a	`\x -> (x, x)`	`a -> (a, a)`
b	`map (+1)`	`Num b => [b] -> [b]`
c	`filter even`	`Integral a => [a] -> [a]`
d	`\f x -> f (f x)`	`(a -> a) -> a -> a`
e	`zip "ab"`	`[b] -> [(Char, b)]`
f	`(.) not`	`(a -> Bool) -> a -> Bool`
g	`head . tail`	`[a] -> a`
h	`\x y -> x == y && x < 3`	`(Ord a, Num a) => a -> a -> Bool`

Marking: 1 each. Constraints must be there for (b), (c), (h); `Eq` isn't needed in (h) because `Ord` includes it. Letter names (`a`, `b`, `t`) don't matter.

Question 2: Evaluation

a) 3. `5 - (3 - (1 - 0))` = `5 - (3 - 1)` = `5 - 2` = 3 (foldr groups from the right).

b) -9. `((0 - 5) - 3) - 1` = `(-5 - 3) - 1` = `-8 - 1` = `-9` (foldl groups from the left).

c) `[1,2,3,4,6]`. x = 1: y = 1, 2, 3 gives 1, 2, 3. x = 2: y = 2, 3 gives 4, 6.

d) `"fp"`

e) `[(1,'a'),(2,'b'),(3,'c')]` (zip stops at the shorter list, so the infinite `[1 ..]` is fine).

f) `[2,12]`

g) 17. `1 + 2 * f [2,3]` = `1 + 2 * (2 + 2 * f [3])` = `1 + 2 * (2 + 2 * (3 + 2 * f []))` = `1 + 2 * (2 + 2 * 3)` = `1 + 2 * 8` = 17.

h) `"1223"` (`show` turns each number into a string; `concatMap` joins them).

Marking: 1 each; for (a), (b), (g) the steps are needed for the mark.

Question 3: Lists and recursion

```haskell
count :: Eq a => a -> [a] -> Int
count _ [] = 0
count y (x:xs)
```

```
  | x == y    = 1 + count y xs
  | otherwise = count y xs
```

[3: base case 1, matching case 1, other case 1]

```
interleave :: [a] -> [a] -> [a]
interleave [] ys     = ys
interleave (x:xs) ys = x : interleave ys xs
```

[3: base case 1, taking x 1, swapping the lists 1. A version with three cases (both non-empty, then each empty) is also fine.]

```
runLengths :: Eq a => [a] -> [(a, Int)]
runLengths []     = []
runLengths (x:xs) = (x, 1 + length (takeWhile (== x) xs)) : runLengths (dropWhile
 (== x) xs)
```

[4: base case 1, counting the run 1, skipping the run 1, recursion on the rest 1]

Question 4: Higher-order functions

```
lengthF :: [a] -> Int
lengthF = foldr (\_ n -> n + 1) 0
```

[3: foldr 1, the function 1, starting value 0 1]

```
applyAll :: [a -> a] -> a -> a
applyAll = foldr (.) id
```

[5: full marks for this. A recursive answer `applyAll [] x = x; applyAll (f:fs) x = f (applyAll fs x)` gets 4.]

Why it works: `foldr (.) id [f, g]` is `f . (g . id)`, so `g` (the last function) runs first. `id` is the function that changes nothing.

Question 5: Data types and trees

```
area :: Shape -> Double
area (Circle r) = pi * r * r
area (Rect w h) = w * h
```

[3: one equation per constructor, correct formulas]

```
values :: BT a -> [a]
values Empty        = []
values (Fork x l r) = values l ++ [x] ++ values r

maxBT :: Ord a => BT a -> Maybe a
maxBT t = case values t of
  []  -> Nothing
  vs  -> Just (maximum vs)
```

[3: collecting the values 1, the empty case 1, Just the maximum 1]

```haskell
paths :: BT a -> [[a]]
paths Empty                = []
paths (Fork x Empty Empty) = [[x]]
paths (Fork x l r)         = [x : p | p <- paths l ++ paths r]
```

[4: leaf case 1 (it must come before the general case), empty case 1, adding x to every path 1, both subtrees 1]

Question 6: Proof by induction

Base case (`xs = []`):

```
length ([] ++ ys)
= length ys                  -- (A1)
= 0 + length ys              -- arithmetic
= length [] + length ys      -- (L1)
```

Induction hypothesis: for some list `xs`, `length (xs ++ ys) = length xs + length ys`.

Step case (`x:xs`):

```
length ((x:xs) ++ ys)
= length (x : (xs ++ ys))        -- (A2)
= 1 + length (xs ++ ys)          -- (L2)
= 1 + (length xs + length ys)    -- induction hypothesis
= (1 + length xs) + length ys    -- + is associative
= length (x:xs) + length ys      -- (L2), backwards
```

So it holds for every finite list `xs`, by induction.

Marking: base case 2, hypothesis stated 1, step case 3 (one mark lost per missing or wrong justification).